

FruitNeRF: A Unified Neural Radiance Field based Fruit Counting Framework

Lukas Meyer¹, Andreas Gilson^{2,3}, Ute Schmid³ and Marc Stamminger¹

Abstract—We introduce *FruitNeRF*, a unified novel fruit counting framework that leverages state-of-the-art view synthesis methods to count any fruit type directly in 3D. Our framework takes an unordered set of posed images captured by a monocular camera and segments fruit in each image. To make our system independent of the fruit type, we employ a foundation model that generates binary segmentation masks for any fruit. Utilizing both modalities, RGB and semantic, we train a semantic neural radiance field. Through uniform volume sampling of the implicit Fruit Field, we obtain fruit-only point clouds. By applying cascaded clustering on the extracted point cloud, our approach achieves precise fruit count. The use of neural radiance fields provides significant advantages over conventional methods such as object tracking or optical flow, as the counting itself is lifted into 3D. Our method prevents double counting fruit and avoids counting irrelevant fruit. We evaluate our methodology using both real-world and synthetic datasets. The real-world dataset consists of three apple trees with manually counted ground truths, a benchmark apple dataset with one row and ground truth fruit location, while the synthetic dataset comprises various fruit types including apple, plum, lemon, pear, peach, and mango. Additionally, we assess the performance of fruit counting using the foundation model compared to a U-Net.

I. INTRODUCTION

Due to a steadily growing global population [1], a declining workforce in several industrialized nations, and the advancement of climate change [2], the field of Precision Agriculture (PA) has seen a significant increase in both, application and research, in recent years [3].

In PA, fruit counting is crucial for obtaining precise yield estimates to optimize harvest, and post-harvest management [4]. However, fruit counting remains a challenging task due to the need for accurate detection and tracking of fruits across multiple images [5] or in combination with 3D point clouds [6], regardless of visibility issues, partial occlusion, or varying lighting conditions. In this context, it is crucial to prevent double counting and ensure that irrelevant fruit, such as fallen fruit or background fruit, are not erroneously included in the count. Additionally, it is challenging to use the same counting method across different fruit types and environments.

In our work, we propose *FruitNeRF*, a novel unified fruit counting framework based on Neural Radiance Fields (NeRF) [7]. The chosen architecture is inherently agnostic to the type of fruit and thus, provides the technical foundation for a generalized fruit counting approach.

The authors from ¹ are with Visual Computing Erlangen (VCE), Friedrich-Alexander-Universität Erlangen-Nürnberg-Fürth, Germany, ² is with the Fraunhofer Institute for Integrated Circuits (IIS) - EZRT, Fürth, Germany and ³ are with Cognitive Systems, University of Bamberg, Germany E-Mail: [lukas.meyer, marc.stamminger]@fau.de, andreas.gilson.fraunhofer.iis.de, ute.schmid@uni-bamberg.de



Fig. 1: Rendering of an apple tree from our real-world dataset generated with *FruitNeRF*. On the right: a zoomed-in region with corresponding semantic logits (top) and the appearance rendering (bottom).

In the first phase, semantic image masks are calculated specific to the type of fruit under consideration. Combining the foundation models DINO [8] and Segment Anything (SAM) [9], fruit masks for all posed images are generated. In comparison, we examine a specialized neural network, U-Net [10], that was trained specifically on apples. The subsequent step of our framework involves optimizing a semantic neural radiance field denoted as *FruitNeRF*, utilizing both RGB and semantic masks to encode the spatial information of fruits within a neural radiance field. In the third stage, we uniformly sample the density and semantic fields (rendered in Fig. 1) of the NeRF to acquire a point cloud that exclusively captures 3D points attributed to fruits. In the last stage, the fruit point cloud is clustered, resulting in a precise fruit count.

We evaluate our framework with both synthetic and real-world data and demonstrate that *FruitNeRF* generalizes well across different fruit types. The main contributions of our work are:

- We propose a novel fruit counting method from unordered images utilizing semantic NeRFs.
- We release a fruit dataset comprising synthetic data from various fruit trees and a real-world dataset specifically focused on apple trees¹.
- The code of *FruitNeRF*² has been made open-source.

¹Project website: https://meyerls.github.io/fruit_nerf

²FruitNeRF code: <https://github.com/meyerls/FruitNeRF>

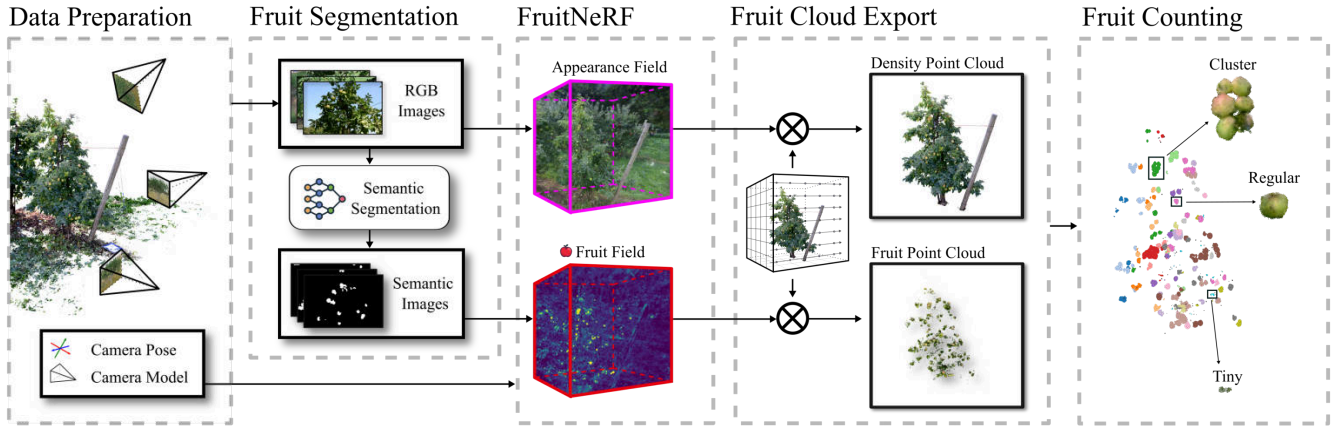


Fig. 2: Pipeline of our proposed fruit counting method - *FruitNeRF*. Data Preparation (Sec. III-A) uses structure from motion (SfM) [20] to recover both intrinsic and extrinsic camera parameters. We then extract semantic masks for arbitrary fruit types (Sec. III-B) using the foundation model SAM [9] and a self-trained U-Net [10] for apples only. The posed RGB and semantic images are used to train a semantic neural radiance field. *FruitNeRF* (Sec. III-C) encodes the appearance of the scene including the semantic information. By sampling the appearance and Fruit Field (Sec. III-D) uniformly, a dense point cloud is obtained. This paves the way for selecting only the 3D fruit points and clustering them to achieve a precise fruit count (Sec. III-E).

II. RELATED WORK

Recent advances in computer vision and hardware have made monitoring fruits a feasible task in horticulture. A strong focus in this connection is laid on sweet peppers [11], [12], strawberries [13], [14], tomatoes [12], and apples [5], [15], [16], [17]. Therefore, we present a short overview of fruit counting methods. In literature, fruit counting pipelines are commonly split into two distinct tasks: fruit detection, and fruit tracking and counting.

While the detection stage was dominated by hand-crafted features (e.g., color-based, shaped-based, etc.) [4] in the past, with the rise of deep learning this task has been successfully replaced by a vast variety of segmentation or detection network architectures (e.g., VCC, ResNet, YOLO) [4].

Fruit tracking and counting on the other hand are still challenging. The main causes of errors are double counting and counting fruit outside the region of interest (e.g., fallen fruit or fruit from trees in back rows). Especially, double counting arises from various sources, such as observing the same fruit in two consecutive images or counting fruit from both sides. To address this issue, researchers have proposed the following strategies [4]: counting fruits only on a per image level [18], tracking fruits across successive frames [5], [15], leveraging sparse point cloud data to count the fruit location in space [17], [6], [16] or projecting 2D instance segmentation onto 3D space [19].

The work from Liu *et al.* [5] is the first to cover the entire automatic fruit counting pipeline applied to an image sequence. In the detection phase, they employ a network architecture, which segments each image into fruit and non-fruit pixels. To assign masks across multiple consecutive frames, they use a Kalman Filter-corrected optical flow tracker. Additionally, they localize fruit locations in 3D by tracking image features across frames to rectify counting errors such as double counting or detecting background and ground fruit.

Häni *et al.* [16] present a modular end-to-end counting system in apple orchards, which connects multiple components from previous work regarding fruit detection [17], counting [17] and tracking [6]. In their approach, fruit detection and counting are merged by determining the fruit number on a per-image level. They compute a semantic point cloud by projecting the point cloud back to all camera frames and computing the intersection with the segmentation mask to identify 3D points belonging to apples [6]. The point clouds are afterward clustered and projected clusterwise to the image plane to obtain the number of apples in a cluster [17].

Gené-Mola *et al.* [19] compute the instance segmentation mask for all images using a convolutional neural network. To obtain a semantic point cloud they use Structure from Motion (SfM) [20] with masked images and to cluster the corresponding point cloud. By back-projecting the clusters into multiple images they assign each cluster an instance ID.

However, the discussed approaches [5], [16], and [19] are all combining SfM and 2D segmentation masks to compute the semantic point cloud and the position of the apples in space. In comparison with *FruitNeRF*, we first perform a semantic reconstruction and then perform the fruit counting in 3D resulting in improved reliability.

III. PROPOSED APPROACH

In this chapter, we introduce the methodology and pipeline for *FruitNeRF* which is depicted in Fig. 2.

A. Data Preparation

The initial step for our pipeline is data preparation. Both our synthetic and real-world datasets consist of sets of RGB images. A detailed description of the generated and recorded data is provided in section IV-A.

For the unordered image data, camera poses for all corresponding images and camera intrinsic parameters are recovered, which are both obtained by SfM [20].

B. Fruit Segmentation

For fruit segmentation, two different methods are considered. The first is a fruit-agnostic foundation model, which offers a generalized solution applicable to all types of fruits. We compare this approach to a supervised neural network that has been fine-tuned specifically for apple segmentation.

1) *Unified Fruit Model*: For the unified fruit segmentation model we used Grounded-SAM [21]. It combines the open-set object detector Grounding DINO [22] and the open-world segmentation model SAM [23]. Grounding DINO generates precise bounding boxes for every image by leveraging textual information as an input condition. The computed bounding boxes are then used by SAM as a box prompt to predict accurate segmentation masks. The advantage of this approach is that it works without fine-tuning or labeling new data.

2) *Dedicated Fruit Model*: In direct comparison, a U-Net [10] was trained on supervised data for apple segmentation. Utilizing our new data combined with the Fuji-SfM dataset [30], an apple dataset was crafted and then used for training. More information on the dataset can be found in Sec. IV-A.2 and training details are listed in Sec. IV-B.1.

C. FruitNeRF

FruitNeRF is the core part of the pipeline. Classic 3D representations such as point clouds, voxels, and SDFs are limited in their ability to represent fine details in complex typologies efficiently. Thus, NeRF, with its implicit volumetric representation, can store multi-view consistent multi-modal scenes such as appearance and semantic data. The network structure of *FruitNeRF* is depicted in Fig. 6.

1) *Volumetric Rendering*: NeRF [7] optimizes a neural radiance field by using a set of posed images and a camera model. The scene itself is implicitly represented by a multi-layer perceptron (MLP). The network learns how to map a spatial coordinate point $\mathbf{x} \in \mathbb{R}^3$ and a view direction $\mathbf{d} \in \mathbb{S}^2$ to a volume density σ and an RGB radiance $\mathbf{c} = (r, g, b)$. The density Field $\mathcal{F}_\sigma : \mathbf{x} \rightarrow \sigma$ is a function of the 3D position and the appearance field $\mathcal{F}_c : (\mathbf{x}, \mathbf{d}) \rightarrow \mathbf{c}$ is a function of the 3D position $\mathbf{x}$ and view direction $\mathbf{d}$.

The color $\mathbf{c}$ of a pixel is computed by querying the MLP at sample points along a camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$. The ray originates in the camera $\mathbf{o}$ and its direction is determined by the pixel position. The estimated color $\hat{\mathbf{C}}(\mathbf{r})$ for one pixel is then computed by accumulating density and color values along K sampled points over the ray $\mathbf{r}$ using volumetric rendering:

$$\hat{\mathbf{C}}(\mathbf{r}) = \sum_{k=1}^K \hat{T}(t_k) \alpha(\sigma(t_k) \delta_k) \mathbf{c}(t_k), \quad (1)$$

$$\text{where } \hat{T}(t_k) = \exp\left(-\sum_{a=1}^{k-1} \sigma(t_a) \delta_a\right).$$

$\delta_k = t_{k+1} - t_k$ is defined as distance between two adjacent sampled points and $\alpha(x) = 1 - \exp(-x)$ as the transmittance probability. An RGB rendering of a fruit tree can be seen on the left side of Fig. 3a.

2) *Semantic Rendering*: The idea of *FruitNeRF* is to encode semantic information about the fruit in 3D. We extend NeRF, similar to the work of Zhi *et al.* [24], by learning to map a 3D point not only to density and color but also to extend it to semantics. Therefore, an additional MLP is defined, which can be seen as a semantic field $\mathcal{F}_s : \mathbf{x} \rightarrow s$ that approximates the semantic logits as a function of only the 3D position $\mathbf{x}$. To approximate the estimated semantic logit $\hat{\mathbf{S}}(\mathbf{r})$ for a pixel, we accumulate the density and semantics along K points sampled over the ray $\mathbf{r}$ using numerical quadrature:

$$\hat{\mathbf{S}}(\mathbf{r}) = \sum_{k=1}^K \hat{T}(t_k) \alpha(\sigma(t_k) \delta_k) \mathbf{s}(t_k). \quad (2)$$

A semantic rendering of a fruit tree can be seen on the left side of Fig. 3a.

D. Point Cloud Export

FruitNeRF incorporates the spatial information of fruits within its density field. To effectively utilize this, the *FruitNeRF* volumes are sampled to process the resulting point cloud further. The *FruitNeRF* model comprises the Density, Appearance, and Fruit Fields as depicted in Fig. 6. The Density Field encodes the density of a point in space, independent of whether it pertains to the trunk, foliage, ground, or fruit. The Appearance Field encodes the corresponding color value, while the Fruit Field contains semantic information regarding the presence of a fruit. Sampling solely from the Fruit Field would result in a scattered point cloud as during training the semantic information updates along ray and smears the information also in empty space. To address this, we link the semantic points with the density and allow only points with a certain density to be included in the resulting point cloud. Fig. 3b shows the extracted point cloud from the Density and Appearance Field (left) and the fruit point cloud (right).

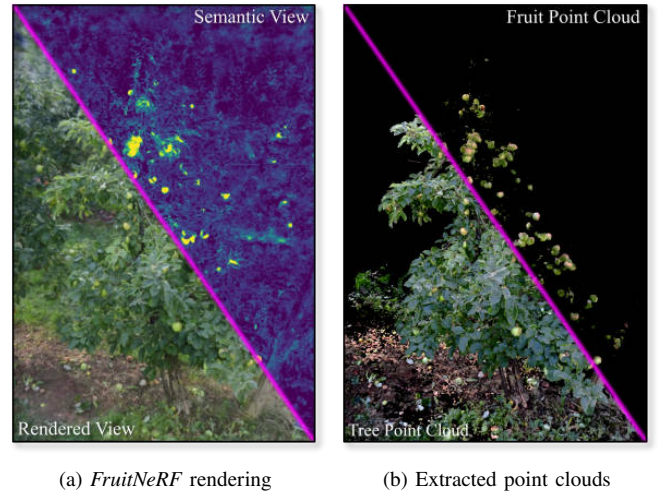


Fig. 3: Visualization of data points along the pipeline. In (a) RGB and semantic rendering are depicted. (b) shows the extracted tree point cloud from the density and the Appearance field (left) and the fruit point cloud, a combination of the Density and the Fruit Field (right).

Afterward, the point cloud is manually cropped to include only the tree of interest, as we aim to obtain a per-tree fruit count evaluation.

E. Fruit Counting

To enumerate individual fruits within the extracted fruit point cloud, we have developed a cascaded two-stage clustering methodology. The initial stage undertakes coarse clustering, while the subsequent stage refines this clustering process to detect invalid and multiple fruit.

Before clustering, we pre-process the point cloud by removing noise. This involves filtering points within a specified radius if they lack a minimum number of neighboring points.

In the first stage of clustering, we utilize density-based spatial clustering (DBSCAN) [25]. This method offers the advantage of not requiring prior knowledge of the number of clusters present. Instead, it identifies clusters based on the density of closely packed points, defining clusters as regions with a high concentration of data points. Consequently, we identify three types of clusters: single, multi, and tiny. The clusters are visualized in Fig. 2. Single-fruit clusters are directly assigned to the count through their clear identification by a similar volume to the template fruit. Multi-fruit clusters contain more than one fruit in a packed vicinity and are identified through oversized volume. Tiny fruit clusters are small in volume and may represent noise from erroneous segmentation or fruits captured from only a sparse set of viewpoints.

Before proceeding to the second clustering stage, we examine the set of tiny fruit clusters. If the distance between neighboring cluster centers is smaller than the average radius of a fruit, these clusters likely represent the same fruit, and thus, are merged. For the remaining tiny fruit clusters, we assess if their volume is similar to the expected volume of a target fruit. If not, we discard the cluster.

The second clustering stage aims to determine the quantity of the multi-fruit clusters. E.g., specific for apples, a reasonable upper bound for the number of apples within a cluster is $N = 6$ [17]. A multi-fruit cluster is identified if the volume of a cluster exceeds the size of our template fruit. In the second stage, we employ agglomeration clustering [26], a hierarchical clustering method. The point cloud undergoes clustering multiple times, with a predefined cluster size ranging from 1 to N . For each clustering result, we compute the cluster center and overlay a template point cloud of the fruit. Subsequently, we compute the maximum mismatch between two point sets with the Hausdorff distance [27] through

$$d_{HD}(\mathcal{X}, \mathcal{Y}) = \sup \left\{ \sup_{x \in \mathcal{X}} \inf_{y \in \mathcal{Y}} d(x, y), \sup_{y \in \mathcal{Y}} \inf_{x \in \mathcal{X}} d(x, y) \right\}. \quad (3)$$

It computes the distance between the point cloud of the template fruit, $\mathcal{X}$, and the cluster point cloud hull, $\mathcal{Y}$. We then determine the minimal distance $d_{min} = \min(d_{HD}^1, \dots, d_{HD}^{N-1}, d_{HD}^N)$ and choose the corresponding cluster size. The steps are visualized in Fig. 4.

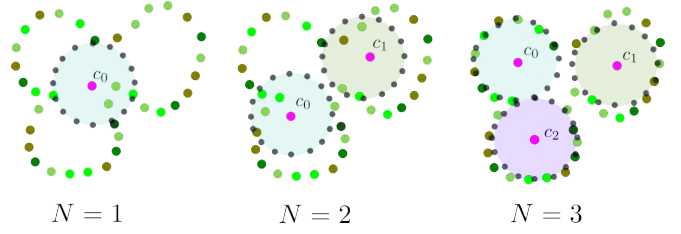


Fig. 4: Second clustering stage: For multi-fruit cluster with three fruits, we simultaneously compute several cluster sizes of the fruit point cloud (greenish dots). Each computed cluster center c (magenta dots) serves as the center point for our template fruit (smaller black dots). The minimum Hausdorff distance between the template point cloud and the fruit point cloud determines the number of clusters.

IV. EXPERIMENTS AND RESULTS

A. Dataset

For our experiments, we generated a series of synthetic scenes featuring fruit trees, complemented by recordings of three real-world apple trees from an orchard setting. The data has been made publicly available, and visualizations can be accessed on the project website.

1) *Synthetic Blender Dataset*: The synthetic dataset was generated using Blender, with various fruit tree models, such as apple, plum, lemon, pear, peach, and mango trees, sourced from XFrog [28]. Each tree was rendered individually using the BlenderNeRF plugin [29]. This plugin facilitated the placement of a virtual camera on a hemisphere, allowing us to extract both the camera’s extrinsic and intrinsic parameters for randomly sampled perspectives directed toward the tree.

For the virtual camera, we opted for a focal length of 35 mm and set the image size to $1024 \text{ px} \times 1024 \text{ px}$. In addition to



(a) Synthetic dataset - apple, pear, plum, mango, lemon, peach



(b) Real-world dataset (rendered) - Tree 03, 02, 01

Fig. 5: Visualization of the synthetic data rendered with Blender (a) and the three apple trees of our real-world dataset (b).

rendering the photometric images, we also generated semantic masks for each fruit tree. Furthermore, we extracted masks from Grounded-SAM for every image. For every fruit tree, a total number of 300 images were rendered. A visualization of all fruit trees is depicted in Fig. 5a.

2) *Real World Dataset*: Recordings for the real-world dataset were conducted at the Hiltpoltstein Fruit Information Center in Bavaria, Germany. We selected three apple trees of the *Resista* variety, and their pruning closely resembles traditional fruit-growing methods. A visualization of the apple trees is shown in Fig. 5b.

The dataset was captured using a Nikon D7100 DSLR camera with a lens featuring a focal length of 35 mm. The captured images have a resolution of 4000 px $\times$ 6000 px. We captured approximately 350 images per tree from a consistent distance of 3 meters, covering multiple heights on both sides of the tree. For recovering the poses of the images, we utilized COLMAP [20]. All apples on each tree were manually counted to obtain a per-tree ground truth. The apple count data is summarized in Figure 8. For the real-world dataset, we also provide the predicted masks from both Grounded-SAM and U-Net.

B. Implementation Details

1) *Fruit Segmentation*: To generate fruit-specific segmentation masks, we employed Grounded-SAM [21] with pre-trained weights. In our pursuit of optimal mask output, we experimented with various text prompts to enhance segmentation mask quality and effectively identify different fruits within each dataset. Overall, we achieved satisfactory results across various fruit types using the generic text prompt "fruits". However, for specific fruits such as apple, plum, lemon, pear, and peach, employing the fruit's name as the text prompt yielded the most accurate results. For mangos, we did not obtain good results in both cases, but instead for using the prompt 'apple'. It should be noted that using the singular of the fruit name achieved significantly better results than the plural. Furthermore, to increase the number of segmentation results, we set the detection threshold in grounding DINO and SAM to a low-threshold value. Additionally, for the real-world dataset we segmented the images on the maximum image size. For the computation of *FruitNeRF* we down-sampled the images and semantic masks to a resolution of 1000 px $\times$ 1500 px.

Our second network is a self-trained U-Net, using a manually annotated subset of 62 images from our real-world data. These images were tiled into smaller 2000 px $\times$ 2000 px sub-images and resized by a factor of 0.5 to meet GPU memory constraints. To enhance the dataset, we incorporated 2D images and segmentation masks from [30] and employed a random-seeded augmentation pipeline that includes geometric transformations, color distortions, and pixel dropout. A PyTorch implementation of U-Net [31] was trained for the task of binary segmentation of apples.

One key difference between masks generated with SAM and U-Net is that SAM produces masks with soft edges that do not precisely align with the image edge e.g., between

apples and leaves. Conversely, U-Net produces masks that closely resemble the original image, offering a more accurate representation. Both architectures were evaluated using the validation split from the fuji dataset [30] as a holdout test set. Segmentation results were measured using intersection over union weighted on class prevalence per image. Segmentation results averaged over all test set images were 0.919 for SAM and 0.962 for U-Net. The results of *FruitNeRF* based on Grounding DINO and SAM masks compared to the U-Net results for apple counting are displayed in Fig. 8.

2) *NeRF Implementation and Training*: The basis for *FruitNeRF* is Nerfacto [32], a method that combines state-of-the-art components from recent papers with significant impact regarding ray generation and sampling, scene contraction, and NeRF fields.

For *FruitNeRF* we extended Nerfacto by a semantic component, also referred to as Fruit Field. The overview of the NeRF architecture is depicted in Fig. 6. The semantic branch takes only the feature vector of the predicted density as an input and not the viewing direction, as the semantic modality is view-independent. For training on the posed images, we used a default rendering loss to compute the photo-metric error between the pixel's RGB value $\mathbf{C}(\mathbf{r})$ and the predicted color value $\hat{\mathbf{C}}(\mathbf{r})$ for ray $\mathbf{r}$ by:

$$\mathcal{L}_{\text{Photo}} = \frac{1}{|\mathcal{R}|} \sum_{\mathbf{r} \in \mathcal{R}} \|\mathbf{C}(\mathbf{r}) - \hat{\mathbf{C}}(\mathbf{r})\|_2^2, \quad (4)$$

where $\mathcal{R}$ is denoted as the set of sampled rays. Regarding the semantics, we leveraged binary cross entropy for pixel-wise classification probability in fruit or background class by:

$$\mathcal{L}_{\text{Sem}} = \frac{1}{|\mathcal{R}|} \sum_{\mathbf{r} \in \mathcal{R}} p(\mathbf{r}) \log \hat{p}(\mathbf{r}) + (1 - p(\mathbf{r})) \log(1 - \hat{p}(\mathbf{r})). \quad (5)$$

The total training loss is then composed by:

$$\mathcal{L} = \mathcal{L}_{\text{Photo}} + \mathcal{L}_{\text{Sem}}. \quad (6)$$

In semantic NeRF [24] the semantic loss gets an additional weight as they propagate the semantic gradient back through the Density Field. We on the other side restrict back-propagation of the semantic loss to only the Fruit Field.

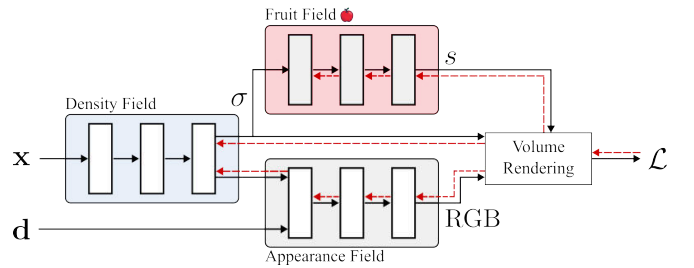


Fig. 6: Overview of the *FruitNeRF* architecture, which is split up into three different components. The density field encodes the volume density σ , the Appearance Field the color RGB , and the Fruit Field the semantic information about the fruit in space. The dashed red arrow indicates the flow direction of the gradient. The figure is inspired by semantic NeRF [24] and adapted from Özer *et al.* [33].

TABLE I: Detected fruit on different fruit types. The used synthetic data have an image size of 1024 px $\times$ 1024 px and contain 300 frames sampled from the upper hemisphere. The data is listed with rendered GT semantic masks and with masks generated by SAM.

Fruit Type	GT Mask	Precision	Recall	F1-Score	SAM	Precision	Recall	F1-Score	Text prompt
Apple	283/283	1.0	1	1	282/283	0.992	0.989	0.991	apple
Plum	651/781	0.973	0.812	0.885	315/781	1.0	0.403	0.575	apple & plum
Lemon	316/326	0.993	0.963	0.978	326/326	0.982	0.982	0.982	lemon
Pear	236/250	1.0	0.944	0.971	229/250	1.0	0.916	0.956	pear
Peach	148/152	1.0	0.973	0.987	148/152	1.0	0.973	0.987	peach
Mango	926/1150	0.978	0.788	0.873	807/1150	0.989	0.694	0.816	apple

Otherwise, the Density Field would focus on predicting density values for spatial points only belonging to fruit.

We implemented two different network sizes: *FruitNeRF* and *FruitNeRF-Big*. *FruitNeRF* utilizes 2 layers for its Fruit Field, with a hidden layer size of 64 neurons. The input layer has a dimension of 15 (the input is a 15-dimensional latent vector from the Density Field). The output dimension of the neural network is 64, which serves as input for a neural segmentation head that reduces the dimension to a single class. For *FruitNeRF-Big*, we increased the layer depth to 3 and increased the hidden dimension to 128 neurons. The input dimension is set to 30 and the output size is kept identical. In the larger variant, we also increased the overall capacity for the density and appearance according to the Nerfacto-Big implementation [32]. For training, we used an Nvidia RTX A5000 with 24GB VRAM. Training time for *FruitNeRF* is an estimate of 12 min and for *FruitNeRF-Big* roughly 2h and 30min. We used an input image size of 1000 px $\times$ 1500 px for real-world data and 1024 px $\times$ 1024 px for synthetic data.

3) *Field Export*: A sampling of the *FruitNeRF* volume is achieved by viewing the scene with an orthographic camera model. We first define a unit cube or use a predefined region of interest around our scene and select one side of the cube to be the image plane. By splitting the image plane into pixels, we can cast multiple rays and query the *FruitNeRF* at a predefined number of steps. By discarding points with a density and semantic value under a fixed threshold, we obtain a point cloud with only fruit points.

C. FruitNeRF Evaluation and Results

In this section, we evaluate our approach using synthetic and real-world data with a focus on the following points:

- The *FruitNeRF* counting performance is evaluated across six distinct types of fruit based on the synthetic dataset.
- The amount and resolution of input images are varied to investigate the influence of these parameters on resulting fruit counts.
- Two scaled *FruitNeRF* architectures are applied to multiple apple datasets and various segmentation pipelines to demonstrate the robustness and practical applicability of our approach in a real-world setting.

For the first experiment, we trained the default *FruitNeRF* model on each type of fruit using 300 images at a resolution of 1024 px $\times$ 1024 px. Afterward, we evaluated the performance using both ground truth masks and masks generated by Grounded-SAM [21]. The summarized results are presented in Table I. In general, *FruitNeRF* with ground truth masks

exhibited an average F1-score of 0.95 compared to SAM-generated masks at 0.88. Particularly for apples, lemons, pears, and peaches, both sets of results closely matched the actual fruit count, resulting in excellent precision and recall values. The worst results with SAM were achieved for plums and mangoes with recall values of 0.4 and 0.69. This performance drop can be attributed to the significantly higher fruit occlusions, which impaired SAM’s prediction quality. SAM masks are more accurate for trees with relatively low fruit counts and fewer occlusions. It can be observed that the centers of the trees were either poorly reconstructed or not present in the fruit cloud at all.

In our second experiment, we evaluated the impact of varying numbers of images and image resolutions. To achieve this, we down-sampled both RGB images and semantic masks from the original apple tree rendering to resolutions of 1024, 512, 256, and 128 px. We initiated the experiment with a consistent set of 5 images with ground truth masks and incrementally introduced additional images up until 100. For each set of images, we trained a new *FruitNeRF*, extracted the fruit point cloud, and applied our fruit count clustering with constant parameters. The evaluation results for different resolutions are presented in Figure 7.

It is evident that *FruitNeRF* struggles to learn a meaningful representation with a sparse number of images, particularly when fewer than 20 images are used. A notable improvement is observed when using 20-30 images with fruit count peaks across all resolutions. This phenomenon is caused by

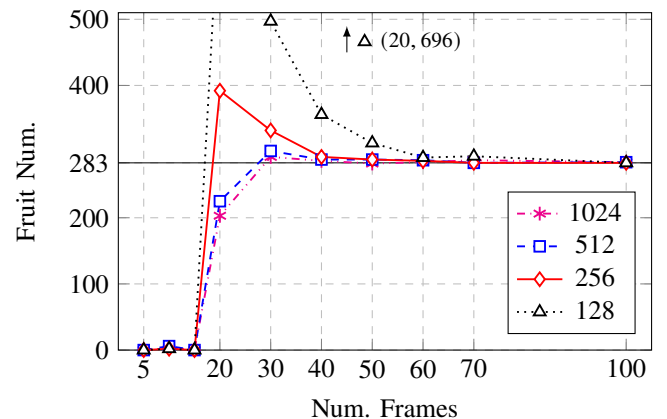


Fig. 7: Fruit count dependence on number of frames. For this evaluation, we utilized the ground truth mask with different sizes. The ground truth count of the synthetic apple tree is 283.

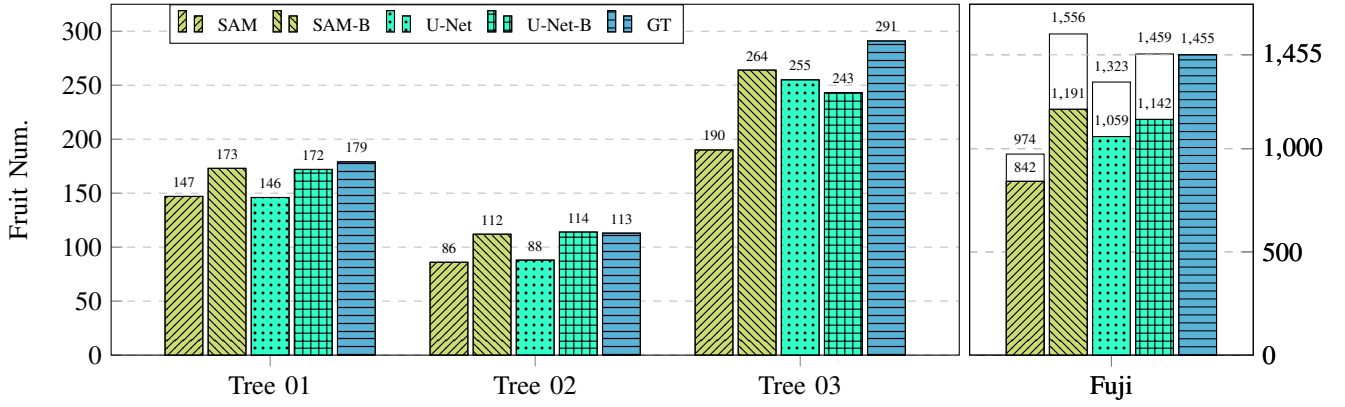


Fig. 8: The estimated apple count for the 3 recorded real-world datasets was evaluated using masks generated with SAM and U-Net, employing two different *FruitNeRF* sizes: default and big (-B). The image size for each tree dataset is 1000 px $\times$ 1500 px. GT apple counts (Tree 01-03) were obtained through manual counting on-site, and clustering parameters were kept consistent per tree. The Fuji dataset [30] contains 11 trees. The highlighted bar indicates the correctly counted (recall), and the white bar on top is the number of overall counted fruit.

insufficient information presented to the Fruit Field, leading to the smearing of semantic knowledge over the entire tree, resulting in a significant increase in clusters falsely counted as fruits. As *FruitNeRF* is exposed to more images, the precision of the fruit cloud improves, as well as the accuracy of the counting. To achieve accurate fruit counts, we found that for a resolution of 128 px, 60 images are required; for 256 px, 50 images are needed; and for 512 px and 1024 px, 30-40 images are necessary.

Lastly, we evaluated *FruitNeRF* using two different real-world datasets. We used masks generated by Grounded-SAM and our U-Net. Additionally, we employed a normal (*FruitNeRF*) and a larger NeRF model (*FruitNeRF-Big*).

For our dataset, depicted in Fig. 8 on the left, we achieved a detection rate on average of $\sim 89\%$ for Trees 1 and 2 for both U-Net and SAM. However, results for Tree 3 reveal a detection rate drop to $\sim 82\%$, which is caused by the tree’s more complex structure and resulting extensive occlusions within the tree crown.

As a second dataset, we choose the Fuji-SfM dataset, which consists of 11 trees in a row captured with 582 images from both tree sides. *FruitNeRF-Big* with SAM and U-Net generated masks achieves a F1-Score of 0.79 and 0.78 respectively, which is near the 0.88 of the original paper [19]. Those are decent results, considering that our pipeline offers unified counting of arbitrary fruits and was not fine-tuned on apples specifically. For the Fuji data, the smaller models performed worse than those with more parameters. The gap between the differently sized architectures can be attributed to the elongated shape of the Fuji scene, which does not make use of the space efficiently. Larger models have increased capacity that can be effectively used to encode data of complex scenes more precisely. In comparison, our recorded apple trees (Tree 01-03) can make better use of the volume within the unit cube as their scenes are less complex and thus, result in smaller performance gaps between *FruitNeRF-Big* and *FruitNeRF*.

V. LIMITATIONS

While *FruitNeRF* demonstrates promising results and addresses common challenges in fruit counting, several limitations persist. The primary constraint lies in the significant training time and GPU memory requirements, as our method heavily relies on image data coverage and quality. Consequently, our approach is not yet suitable for real-time applications or edge computing. Initial experiments with Gaussian Splatting [34] have decreased computing times for our pipeline, and utilizing other architectures, such as PAgNeRF [11], might lead to even faster computations.

Moreover, the cascaded clustering technique involves hyperparameters that require manual adjustment according to the type of fruit, complicating unified automation processes. Additionally, the second clustering stage is bound to a nominal fruit size, necessitating adaptation for valid counting of varying fruit sizes at different growth stages. Implementing a learned approach for fruit detection within the point cloud could significantly alleviate this issue and enhance the efficiency of fruit counting.

As an industrial application, low lighting conditions and different exposure levels due to changing weather conditions could worsen the results of the NeRF reconstruction. Therefore, implementations such as Low-Light NeRF [35] and HDR-NeRF [36] could tackle these problems. Additionally, the presence of a non-static scene, caused by wind, must be further investigated to determine if dynamic NeRF approaches such as RobustNeRF [37] can solve these issues.

Nevertheless, we are confident that the application potential of our approach will benefit from rapid technological improvements in hardware and 3D reconstruction techniques, which will mitigate these limitations.

VI. CONCLUSION AND FUTURE WORK

This paper introduces *FruitNeRF*, a novel framework designed to accurately count visible fruits within a neural radiance field. Leveraging only 2D images, *FruitNeRF* facilitates precise 3D reconstructions of trees, enabling accurate fruit

counting. By integrating Grounded-SAM into the *FruitNeRF* pipeline, arbitrary types of fruit can be counted without the need for costly annotations and the training of a U-Net.

Our framework has been thoroughly validated in both synthetic and real-world scenarios, representing the first application of NeRFs for fruit counting to our knowledge. The experiments demonstrate that *FruitNeRF* achieves an F1-score of 0.95 on our synthetic dataset with ground truth masks, while masks generated by SAM attain around an F1-score of 0.88 averaged over six different fruit species. Additionally, we have illustrated that excellent results can be obtained with only 40 images per tree and a resolution of $512 \text{ px} \times 512 \text{ px}$, demonstrating the scalability and effectiveness of our approach. Fruit counting on our self-recorded real-world apple dataset showcases a detection rate exceeding 89% across various masks and network architectures. For the Fuji benchmark dataset, we demonstrate an F1-score of 0.79.

From this novel fruit-counting approach, several promising directions for future research emerge. Primary efforts should focus on improving the clustering step, as it is highly sensitive to hyper-parameter tuning. Additionally, to broaden the applicability of *FruitNeRF*, the use of time-series images should be considered. This could reduce the number of required images and help achieve real-time performance.

As we aim to be a unified fruit counting network, further investigation into soft fruits such as strawberries, raspberries, grapes, and other small-sized fruits should be carried out.

Extending the framework's utility beyond individual trees to entire orchard rows could be explored through the integration of online pose estimation methods like Simultaneous Localization and Mapping (SLAM). Simulation environments, such as SLAM in Blender [38], could serve as valuable test beds for refining detection rate.

Moreover, the versatility of neural radiance fields extends beyond visible light, presenting opportunities to incorporate other modalities such as near-infrared or thermal imaging, as demonstrated by Özer [33]. *FruitNeRF* demonstrates the potential to enhance fruit counting capabilities, but leveraging neural radiance fields also enables broader orchard analysis, including ripeness assessment, stress level monitoring, and guidance of harvesting robots.

VII. ACKNOWLEDGEMENT

We extend our gratitude to **Adam Kalisz** for his unique Blender skills, **Victoria Schmidt**, and **Annika Killer** for their invaluable assistance in evaluating the recorded apple trees. Additional thanks to **Roland Gruber**, **Christoph Drescher**, and **Julian Bittner** for their annotation support.

This project is funded by the 5G innovation program of the German Federal Ministry for Digital and Transport under the funding code 165GU103B.

REFERENCES

- [1] United Nations, "Shifting Demographics," 2023, Accessed on: Nov. 5, 2023.
- [2] R. Schrijver, *et al.*, "Precision agriculture and the future of farming in Europe," Scientific Foresight Unit - EU, 2016.
- [3] G. Gyarmati, *et al.*, "The present and future of the precision agriculture," SoSE, 2020.
- [4] J. C. Miranda, *et al.*, "Fruit sizing using AI: A review of methods and challenges," *Postharvest Biology and Technology*, 2023.
- [5] X. Liu, *et al.*, "Robust Fruit Counting: Combining Deep Learning, Tracking, and Structure from Motion," *IROS*, 2018.
- [6] P. Roy, *et al.*, "Registering Reconstructions of the Two Sides of Fruit Tree Rows," *IROS*, 2018.
- [7] B. Mildenhall, *et al.*, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," *ECCV*, 2020.
- [8] M. Caron, *et al.*, "Emerging Properties in Self-Supervised Vision Transformers," *ICCV*, 2021.
- [9] A. Kirillov, *et al.*, "Segment Anything," *ArXiv*, 2023.
- [10] O. Ronneberger, *et al.*, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *MICCAI*, 2015.
- [11] S. Claus, *et al.*, "PAg-NeRF: Towards fast and efficient end-to-end panoptic 3D representations for agricultural robotics," *IEEE Robotics and Automation Letters*, 2023.
- [12] C. Smitt, *et al.*, "PATHoBot: A Robot for Glasshouse Crop Phenotyping and Intervention," *ICRA*, 2021.
- [13] A. Riccardi, *et al.*, "Fruit Tracking Over Time Using High-Precision Point Clouds," *ICRA*, 2023.
- [14] M. Sorour, *et al.*, "Compact Strawberry Harvesting Tube Employing Laser Cutter," *IROS*, 2022.
- [15] X. Liu, *et al.*, "Monocular Camera Based Fruit Counting and Mapping With Semantic Data Association," *IEEE Robotics and Automation Letters*, 2019.
- [16] N. Häni, *et al.*, "A comparative study of fruit detection and counting methods for yield mapping in apple orchards," *Journal of Field Robotics*, 2020.
- [17] N. Häni, *et al.*, "Apple Counting using Convolutional Neural Networks," *IROS*, 2018.
- [18] S. Chen, *et al.*, "Counting Apples and Oranges With Deep Learning: A Data-Driven Approach," *IEEE Robotics and Automation Letters*, 2017.
- [19] J. Gené-Mola, *et al.*, "Fruit detection and 3D location using instance segmentation neural networks and structure-from-motion photogrammetry," *Computers and Electronics in Agriculture*, 2020.
- [20] J. L. Schönberger, *et al.*, "Structure-from-Motion Revisited," *CVPR*, 2016.
- [21] T. Ren, *et al.*, "Grounded SAM: Assembling Open-World Models for Diverse Visual Tasks," *ArXiv*, 2024.
- [22] S. Liu, *et al.*, "Grounding dino: Marrying dino with grounded pre-training for open-set object detection," *ArXiv*, 2023.
- [23] A. Kirillov, *et al.*, "Segment Anything," *ArXiv*, 2023.
- [24] S. Zhi, *et al.*, "In-Place Scene Labelling and Understanding with Implicit Scene Representation," *ArXiv*, 2021.
- [25] M. Ester, *et al.*, "A density-based algorithm for discovering clusters in large spatial databases with noise," *Knowledge Discovery and Data Mining*, 1996.
- [26] F. Pedregosa, *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011.
- [27] A. Taha, *et al.*, "An Efficient Algorithm for Calculating the Exact Hausdorff Distance," *PAMI*, 2015.
- [28] XFrog Inc., *LIBRARY: FRUIT TREES*, 2020.
- [29] M. Raafat, "BlenderNeRF" (Version 5.0.0), 2023, [Computer software]. <https://doi.org/10.5281/zenodo.7926211>
- [30] J. Gené-Mola, *et al.*, "Fuji-SFM dataset: A collection of annotated images and point clouds for Fuji apple detection and location using structure-from-motion photogrammetry," *Data in Brief*, 2020.
- [31] A. Milesi, "Pytorch-UNet," *GitHub*, 2017.
- [32] M. Tancik, *et al.*, "Nerfstudio: A Modular Framework for Neural Radiance Field Development," *SIGGRAPH*, 2023.
- [33] M. Özer, *et al.*, "Exploring Multi-modal Neural Scene Representations With Applications on Thermal Imaging," *ArXiv*, 2024.
- [34] B. Kerbl, *et al.*, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *SIGGRAPH*, 2023.
- [35] H. Wang, *et al.*, "Lighting up NeRF via Unsupervised Decomposition and Enhancement," *ICCV*, 2023.
- [36] X. Huang, *et al.*, "Hdr-nerf: High dynamic range neural radiance fields," *CVPR*, 2022.
- [37] S. Sabour, *et al.*, "RobustNeRF: Ignoring Distractors With Robust Losses," *CVPR*, 2023.
- [38] A. Kalisz, *et al.*, "B-SLAM-SIM: A Novel Approach to Evaluate the Fusion of Visual SLAM and GPS by Example of Direct Sparse Odometry and Blender," *VISIGRAPP*, 2019.